



Universidad Veracruzana

Arquitectura de Django

Programación segura

Dr. Juan Manuel Gutiérrez Méndez

Propósito de la sesión

Comprender Django como un framework que organiza una aplicación web mediante capas, convenciones y componentes reutilizables.

Django no solo permite programar páginas web: propone una arquitectura para construir aplicaciones mantenibles, extensibles y seguras.

Al finalizar, el estudiante deberá poder explicar:

- cómo se organiza un proyecto Django;
- cómo fluye una petición web;
- dónde se ubican modelos, vistas, formularios y plantillas;
- qué controles de seguridad aporta el framework;
- por qué Django aplica inversión de control.

Versión de referencia

Para esta unidad se utilizará **Django 6.x** como versión de referencia del curso.

La versión oficial más reciente reportada por el sitio de Django es **Django 6.0.5**.

```
python -m pip install Django==6.0.5
```

En laboratorio, la versión instalada debe mantenerse consistente entre la documentación, el entorno virtual y los ejemplos de código usados en clase.

Django como framework

Django es un framework web de alto nivel para Python, orientado al desarrollo rápido con diseño limpio y pragmático.

En lugar de construir cada componente desde cero, Django proporciona una estructura ya organizada para desarrollar aplicaciones web.

Rutas

Conectan URLs con vistas.

Modelos

Representan datos y relaciones.

Plantillas

Construyen la salida HTML.

Seguridad

Incluye controles integrados.

Componentes que aporta Django

Django proporciona elementos reutilizables para resolver necesidades frecuentes en aplicaciones web.

Componente	Función dentro del framework
Sistema de rutas	Relaciona URLs con vistas.
Vistas	Coordinan la petición y la respuesta.
ORM	Permite trabajar con datos usando objetos de Python.
Formularios	Limpian, convierten y validan entradas.
Plantillas	Renderizan HTML con datos del contexto.
Middleware	Aplica controles transversales.
Autenticación	Gestiona usuarios, sesiones y permisos.
Administración	Proporciona interfaz para gestionar modelos.

Framework vs librería

LIBRERÍA

- Nuestro código decide cuándo usarla.
- La llamamos desde nuestras funciones.
- Resuelve tareas específicas.
- Ejemplo: usar una librería para validar una fecha.

FRAMEWORK

- Define la estructura general.
- Ejecuta partes de nuestro código.
- Controla el ciclo de vida de la aplicación.
- Ejemplo: Django llama nuestras vistas cuando llega una petición.

Inversión de control

En un framework, el flujo principal no lo controla completamente nuestro programa.

La inversión de control ocurre cuando nuestro código se integra al framework y el framework decide cuándo ejecutarlo.

En Django, el estudiante escribe:

- modelos;
- rutas;
- vistas;
- formularios;
- plantillas;
- configuraciones.

¿Qué controla Django?

Django controla el ciclo de vida general de la petición.

Recibe

Atiende la petición HTTP enviada por el navegador.

Resuelve

Identifica la ruta y la vista correspondiente.

Ejecuta

Aplica middleware, sesiones, seguridad y lógica de flujo.

Responde

Devuelve una respuesta HTTP al navegador.

El programador define componentes; Django coordina cuándo y cómo se ejecutan dentro del flujo web.

Proyecto principal y aplicaciones

Un proyecto Django se organiza en dos niveles.

Nivel	Propósito	Ejemplo
Proyecto principal	Configura y conecta el sistema completo.	<code>config/</code>
Aplicaciones Django	Agrupar funcionalidades del dominio.	<code>bienes/</code> , <code>clientes/</code> , <code>direcciones/</code>

El proyecto principal no debe concentrar toda la lógica. Su función es configurar, conectar y habilitar las aplicaciones.

Estructura general del proyecto

```
config/  
  settings.py  
  urls.py  
  asgi.py  
  wsgi.py  
  
apps/  
  bienes/  
    models.py  
    forms.py  
    views.py  
    urls.py  
    templates/  
    admin.py  
    tests.py  
  
  clientes/  
    models.py  
    forms.py  
    views.py  
    urls.py  
    templates/  
  
  direcciones/  
    models.py  
    forms.py  
    views.py  
    urls.py  
    templates/
```

Relación entre proyecto y aplicaciones

config/urls.py

Recibe las rutas principales del proyecto.

apps/bienes/urls.py

Define rutas específicas de la aplicación.

views.py

Coordina el caso de uso solicitado.

forms.py / models.py

Validan entradas y gestionan datos.

templates/

Presenta la respuesta HTML al usuario.

El archivo `config/urls.py` delega hacia cada aplicación; la lógica funcional se mantiene dentro de la app correspondiente.

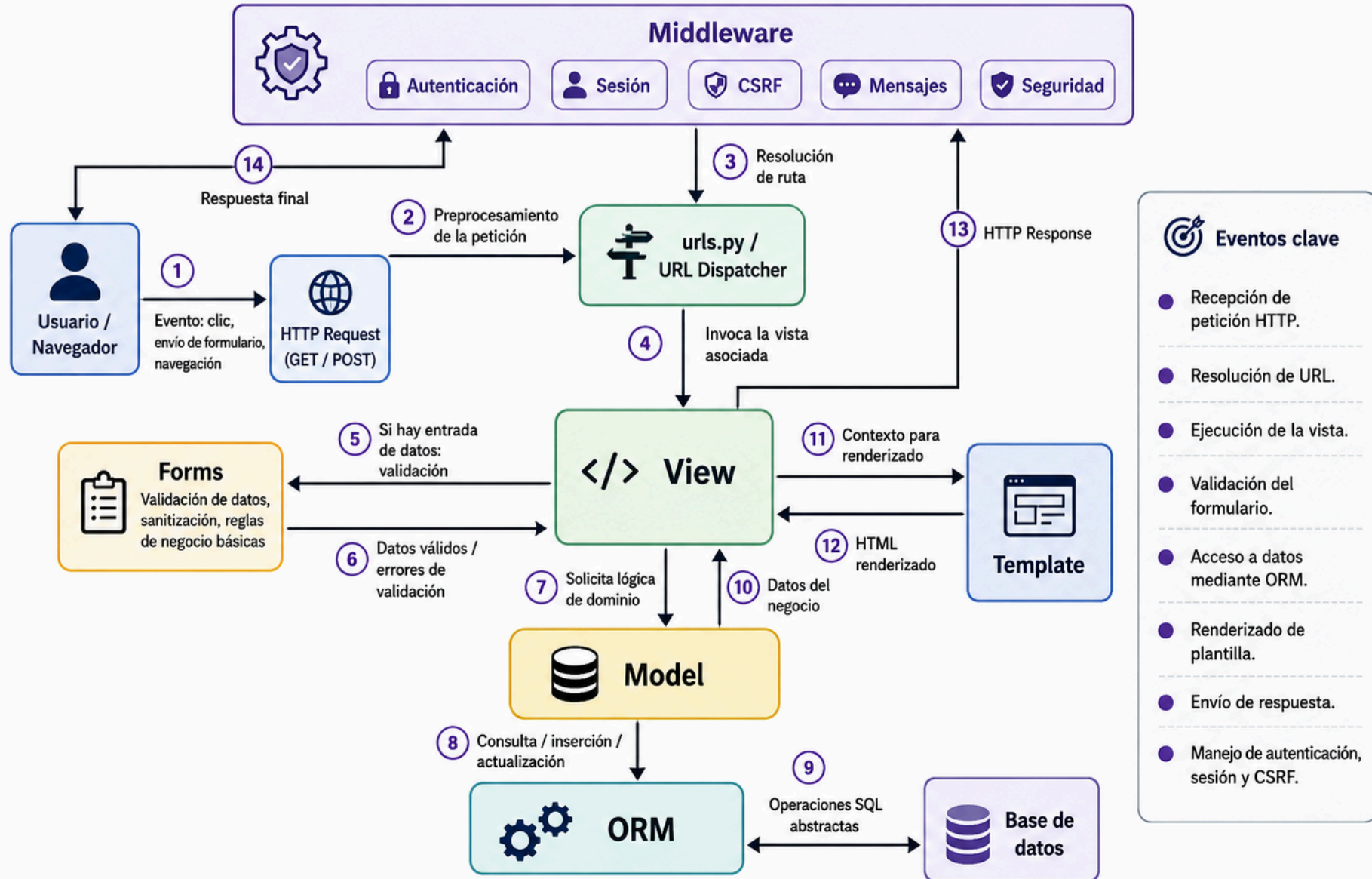
Arquitectura MVT

Django se describe comúnmente mediante el patrón **MVT: Model–View–Template**.

Componente	Responsabilidad principal
Usuario / Navegador	Inicia eventos como clics, navegación o envío de formularios.
HTTP Request	Envía la petición GET o POST hacia Django.
Middleware	Aplica controles transversales: autenticación, sesión, CSRF, mensajes y seguridad.
URL Dispatcher / <code>urls.py</code>	Resuelve la ruta y selecciona la vista correspondiente.
View	Coordina la lógica del flujo: recibe la petición, usa formularios, consulta modelos y devuelve una respuesta.
Form	Limpia, valida y reporta errores sobre los datos enviados por el usuario.
Model	Define la estructura de datos, relaciones y reglas del dominio.
ORM	Traduce operaciones del modelo en consultas hacia la base de datos.
Base de datos	Persiste y recupera la información de la aplicación.
Template	Renderiza la presentación HTML a partir del contexto enviado por la vista.
HTTP Response	Devuelve al navegador el resultado final de la operación.

Arquitectura MVT en Django

Componentes, flujos y eventos



i En Django, la View coordina el flujo; el Model gestiona los datos; el Template presenta la información.

Vista general del flujo MVT

Entrada

El navegador envía una petición HTTP hacia la aplicación Django.

Procesamiento

Middleware, rutas y vistas coordinan la ejecución del caso de uso.

Validación

Los formularios limpian, convierten y validan los datos recibidos.

Datos

El modelo y el ORM gestionan el acceso seguro a la información.

Presentación

La plantilla renderiza la respuesta HTML para el usuario.

En Django, la arquitectura MVT separa la recepción de la petición, la coordinación del flujo, la validación, el acceso a datos y la presentación.

HTTP Request

El **HTTP Request** es la petición que el navegador envía al servidor cuando el usuario interactúa con la aplicación.

Puede originarse al consultar una página, enviar un formulario, iniciar sesión, cargar archivos o solicitar información.

Aspecto	Descripción
Método	Indica la acción solicitada: GET , POST , PUT , DELETE , entre otros.
URL	Ruta que el usuario solicita dentro de la aplicación.
Datos	Parámetros, campos de formulario, archivos o información enviada por el cliente.
Metadatos	Encabezados, cookies, sesión y datos técnicos de la petición.

Desde programación segura, toda petición HTTP debe tratarse como entrada no confiable hasta que sea validada.

HTTP Request en Django

Ubicación dentro del código

En Django, la petición llega a la vista mediante el objeto `request` .

```
def lista_bienes(request):  
    metodo = request.method  
    usuario = request.user  
    return render(request, "bienes/lista.html")
```

Usuario

Realiza una acción.

HTTP Request

Transporta datos y metadatos.

Middleware

Procesa controles transversales.

HTTP Request: criterios de seguridad

Riesgo	Control recomendado
Datos manipulados desde el navegador	Validar siempre en servidor.
Formularios alterados	Usar Forms y validaciones explícitas.
Archivos maliciosos o excesivos	Validar tipo, tamaño y almacenamiento.
Sesiones o cookies comprometidas	Configurar seguridad de sesión y HTTPS.

El navegador puede ayudar a validar, pero la decisión de aceptar datos corresponde siempre al servidor.

Middleware

El **Middleware** es una capa transversal que intercepta la petición antes de llegar a la vista y también procesa la respuesta antes de devolverla al usuario.

No pertenece a una sola aplicación; aplica reglas generales al proyecto.

Función	Propósito
Seguridad	Aplica encabezados y controles globales.
Sesión	Administra datos temporales del usuario.
Autenticación	Asocia la petición con un usuario.
CSRF	Protege formularios contra solicitudes falsificadas.
Mensajes	Permite mostrar avisos temporales al usuario.

El Middleware concentra controles transversales que deben aplicarse de forma uniforme en toda la aplicación.

Middleware en Django

Ubicación dentro del proyecto

El Middleware se configura en `settings.py`.

```
MIDDLEWARE = [  
    "django.middleware.security.SecurityMiddleware",  
    "django.contrib.sessions.middleware.SessionMiddleware",  
    "django.middleware.csrf.CsrfViewMiddleware",  
    "django.contrib.auth.middleware.AuthenticationMiddleware",  
    "django.contrib.messages.middleware.MessageMiddleware",  
]
```

HTTP Request
Petición entrante.

Middleware
Controles
transversales.

URL Dispatcher
Resolución de
ruta.

View
Ejecución del
caso de uso.

Middleware: idea clave

El Middleware permite aplicar políticas comunes sin repetir código en cada vista.

Control	Ejemplo de aplicación
Sesión	Mantener información del usuario autenticado.
Autenticación	Asociar <code>request.user</code> a la petición.
CSRF	Verificar token en formularios POST.
Seguridad	Agregar encabezados y reglas globales.

El Middleware no sustituye las validaciones de la aplicación; las complementa con controles globales.

URL Dispatcher / urls.py

El **URL Dispatcher** es el mecanismo que analiza la URL solicitada y determina qué vista debe atender la petición.

En Django, esta responsabilidad se implementa mediante archivos `urls.py`.

Elemento	Responsabilidad
<code>config/urls.py</code>	Define las rutas principales del proyecto.
<code>apps/<app>/urls.py</code>	Define las rutas específicas de cada aplicación.
<code>path()</code>	Relaciona una ruta con una vista.
<code>include()</code>	Delega rutas hacia una aplicación.

El URL Dispatcher permite que el proyecto principal delegue el flujo hacia aplicaciones específicas.

URL Dispatcher en Django

Rutas principales

```
# config/urls.py
from django.urls import path, include

urlpatterns = [
    path("bienes/", include("apps.bienes.urls")),
    path("clientes/", include("apps.clientes.urls")),
]
```

Rutas de aplicación

```
# apps/bienes/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path("", views.lista_bienes, name="lista_bienes"),
    path("nuevo/", views.crear_bien, name="crear_bien"),
]
```

URL Dispatcher: relación en el flujo

Middleware

Entrega la petición procesada.

URL Dispatcher

Busca el patrón coincidente.

View

Recibe la petición asignada.

Una ruta clara facilita mantenimiento, control de acceso, pruebas y revisión de responsabilidades por aplicación.

View

La **View** coordina el caso de uso que debe ejecutarse ante una petición.

En Django, la View no es la interfaz visual. La presentación corresponde al **Template**.

Responsabilidad	Descripción
Recibir petición	Atiende el objeto <code>request</code> .
Coordinar flujo	Decide si listar, crear, editar, eliminar o consultar.
Usar formularios	Envía datos a <code>forms.py</code> para validación.
Consultar modelos	Solicita información a <code>models.py</code> .
Devolver respuesta	Renderiza HTML, redirige o devuelve errores.

La View coordina el flujo, pero no debe concentrar toda la lógica de validación, persistencia o presentación.

View en Django

Ejemplo básico

```
from django.shortcuts import render, redirect
from .forms import BienForm

def crear_bien(request):
    if request.method == "POST":
        form = BienForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("lista_bienes")
    else:
        form = BienForm()

    return render(request, "bienes/formulario.html", {"form": form})
```

View: relación con otros componentes

URL

Dispatcher

Selecciona la vista.

View

Coordina el caso de uso.

Form

Valida entrada.

Model

Gestiona datos.

Template

Presenta salida.

Una vista debe ser clara, breve y orientada a coordinar componentes, no a concentrar toda la aplicación.

Form

El **Form** recibe, limpia y valida datos enviados por el usuario.

Es una pieza central para programación segura porque controla qué datos pueden avanzar hacia la lógica y la persistencia.

Función	Descripción
Recibir datos	Procesa información enviada por <code>GET</code> o <code>POST</code> .
Limpiar datos	Convierte cadenas en tipos esperados.
Validar reglas	Verifica formato, longitud, rango y obligatoriedad.
Reportar errores	Devuelve mensajes claros para entradas inválidas.
Proteger flujo	Impide que datos incorrectos lleguen al modelo.

El Form funciona como una barrera de validación entre la entrada del usuario y el resto de la aplicación.

Ejemplo con validación

```
from django import forms
from .models import Bien

class BienForm(forms.ModelForm):
    class Meta:
        model = Bien
        fields = ["identificador", "descripcion", "marca", "valor", "estatus"]

    def clean_valor(self):
        valor = self.cleaned_data["valor"]

        if valor <= 0:
            raise forms.ValidationError("El valor debe ser mayor que cero.")

        return valor
```

Form: relación en el flujo

View

Envía datos recibidos.

Form

Limpia, convierte y valida.

View

Recibe datos válidos o errores.

Resultado	Acción
Datos válidos	Continúa el flujo hacia modelo, guardado o respuesta.
Datos inválidos	Regresa errores al usuario mediante la plantilla.

La validación debe realizarse en servidor, aunque también existan controles en el navegador.

Model

El **Model** representa las entidades del dominio y su estructura de datos.

Define campos, tipos, relaciones, restricciones y comportamiento asociado a la información.

Elemento	Ejemplo
Entidad	<code>Bien</code> , <code>Cliente</code> , <code>Direccion</code>
Campo	<code>identificador</code> , <code>descripcion</code> , <code>valor</code>
Restricción	<code>unique=True</code> , <code>max_length</code> , <code>choices</code>
Relación	<code>ForeignKey</code> , <code>OneToOneField</code> , <code>ManyToManyField</code>

El Model no solo representa tablas; también ayuda a proteger la consistencia estructural de los datos.

Ejemplo de entidad

```
from django.db import models

class Bien(models.Model):
    ESTATUS_CHOICES = [
        ("bueno", "Bueno"),
        ("regular", "Regular"),
        ("malo", "Malo"),
    ]

    identificador = models.CharField(max_length=20, unique=True)
    descripcion = models.CharField(max_length=150)
    marca = models.CharField(max_length=80)
    valor = models.DecimalField(max_digits=10, decimal_places=2)
    estatus = models.CharField(max_length=10, choices=ESTATUS_CHOICES)

    def __str__(self):
        return self.identificador
```


Model: relación en el flujo

View

Solicita datos o acciones.

Model

Representa entidades y restricciones.

ORM

Traduce operaciones.

Base de datos

Persiste información.

El modelo ayuda a mantener la integridad estructural mediante tipos, longitudes, restricciones y relaciones.

ORM

El **ORM** es la capa de mapeo objeto-relacional de Django.

Permite trabajar con registros de base de datos mediante objetos de Python, sin escribir SQL manual en la mayoría de los casos.

Operación	Ejemplo
Consultar	<code>Bien.objects.all()</code>
Filtrar	<code>Bien.objects.filter(estatus="bueno")</code>
Crear	<code>Bien.objects.create(...)</code>
Obtener	<code>Bien.objects.get(id=1)</code>
Actualizar	Modificar atributos y ejecutar <code>.save()</code>
Eliminar	Ejecutar <code>.delete()</code>

El ORM reduce riesgos asociados a consultas SQL construidas manualmente, pero no reemplaza la validación ni el control de acceso.

ORM en Django

Ejemplo de uso

```
# Consultar todos los bienes
bienes = Bien.objects.all()

# Filtrar bienes
bienes_buenos = Bien.objects.filter(estatus="bueno")

# Crear un bien
Bien.objects.create(
    identificador="b-0001-2026",
    descripcion="Equipo de cómputo",
    marca="Dell",
    valor=15000,
    estatus="bueno"
)
```

ORM: relación en el flujo

Model

Define entidades.

ORM

Construye operaciones.

Base de datos

Ejecuta persistencia.

Criterio	Implicación
Seguridad	Evita concatenar SQL de forma manual.
Mantenibilidad	Centraliza operaciones a partir de modelos.
Integridad	Trabaja con restricciones definidas en el modelo y la base.

Base de datos

La **base de datos** almacena de forma persistente la información de la aplicación.

En desarrollo puede utilizarse SQLite; en ambientes productivos suelen emplearse gestores como PostgreSQL, MariaDB o MySQL.

Responsabilidad	Descripción
Persistencia	Conserva datos aunque la aplicación se cierre.
Integridad	Aplica restricciones, llaves y relaciones.
Consulta	Permite recuperar información almacenada.
Transacciones	Mantiene consistencia en operaciones relacionadas.
Protección	Requiere permisos, respaldos y controles de acceso.

La base de datos es un activo crítico; debe proteger la confidencialidad, integridad y disponibilidad de la información.

Base de datos en Django

Configuración básica

```
# settings.py
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

ORM

Solicita operaciones.

Base de datos

Persiste y recupera registros.

ORM

Devuelve resultados al modelo.

Base de datos: criterios de seguridad

Aspecto	Control
Acceso	Usuarios y privilegios mínimos.
Respaldo	Copias periódicas y verificadas.
Integridad	Restricciones y migraciones controladas.
Confidencialidad	Protección de datos sensibles.
Disponibilidad	Monitoreo, recursos suficientes y recuperación.

No basta con validar en la interfaz. La base de datos también debe proteger la consistencia y el acceso a la información.

Template

El **Template** es la plantilla que genera la salida HTML que verá el usuario.

Recibe datos desde la View mediante un contexto y los presenta en una estructura visual.

Responsabilidad	Descripción
Presentar información	Muestra datos procesados por la aplicación.
Separar presentación	Evita mezclar HTML con lógica de control.
Mostrar formularios	Renderiza campos, errores y mensajes.
Usar contexto	Recibe variables enviadas por la vista.
Construir interfaz	Organiza tablas, botones, enlaces y formularios.

El Template presenta información, pero no debe concentrar reglas de negocio ni decisiones de seguridad.

Template en Django

Estructura frecuente

```
apps/  
  bienes/  
    templates/  
      bienes/  
        lista.html  
        formulario.html
```

Ejemplo de plantilla

```
<h1>Bienes registrados</h1>

<table>
  {% for bien in bienes %}
    <tr>
      <td>{{ bien.identificador }}</td>
      <td>{{ bien.descripcion }}</td>
      <td>{{ bien.estatus }}</td>
    </tr>
  {% endfor %}
</table>
```

Template: relación en el flujo

View

Envía contexto.

Template

Renderiza HTML.

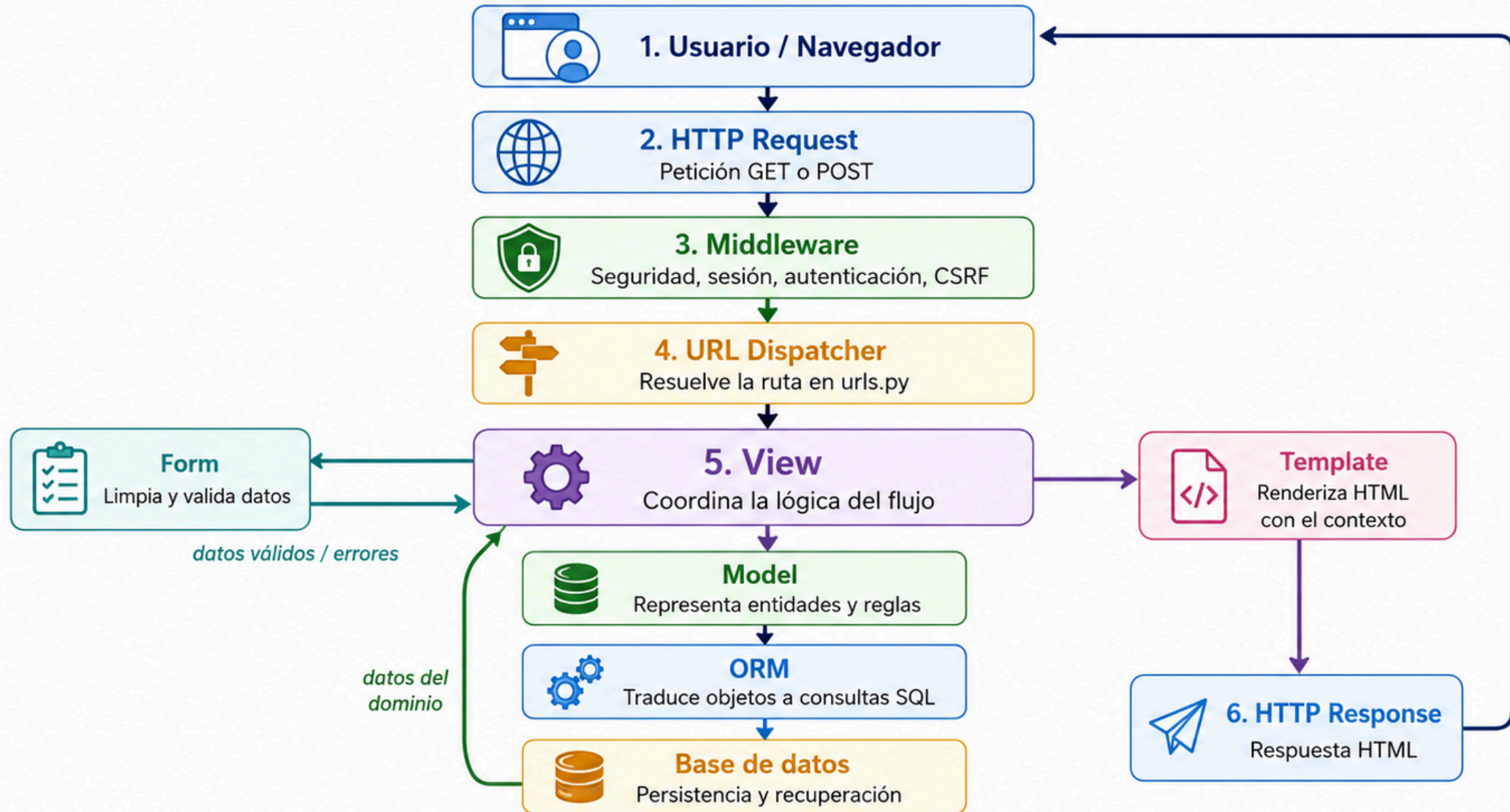
HTTP Response

Devuelve la salida al navegador.

La plantilla debe presentar datos, no decidir reglas de negocio ni resolver validaciones principales.

Flujo de arquitectura MVT en Django

Interacción entre componentes durante una petición web



Idea clave: la View coordina el proceso, el Form valida la entrada, el Model y el ORM gestionan los datos y el Template presenta el resultado.

Síntesis: dónde vive cada responsabilidad

Responsabilidad	Componente	Archivo frecuente
Configuración global	Proyecto principal	<code>config/settings.py</code>
Rutas generales	Proyecto principal	<code>config/urls.py</code>
Rutas funcionales	Aplicación	<code>apps/<app>/urls.py</code>
Coordinación del flujo	View	<code>views.py</code>
Validación de entrada	Form	<code>forms.py</code>
Entidades y restricciones	Model	<code>models.py</code>
Acceso a datos	ORM	<code>Model.objects...</code>
Presentación HTML	Template	<code>templates/</code>

La seguridad se revisa por responsabilidad: entrada, rutas, vista, validación, modelo, persistencia y salida.

Lectura de arquitectura segura

Una aplicación Django puede revisarse desde los puntos donde se reciben, procesan, transforman, almacenan y presentan datos.

01

Entrada

Peticiones, formularios, parámetros, archivos y sesiones.

02

Control

Rutas, vistas, permisos, middleware y flujo de ejecución.

03

Datos

Modelos, restricciones, ORM, migraciones y base de datos.

04

Salida

Plantillas, respuestas HTTP, mensajes y errores visibles.

Cierre conceptual

Django organiza la aplicación mediante convenciones. Esta organización permite ubicar responsabilidades, reducir acoplamiento y revisar controles de seguridad por componente.

Para analizar una aplicación Django, conviene preguntar:

- ¿dónde entra el dato?
- ¿quién lo valida?
- ¿qué vista coordina el flujo?
- ¿qué modelo representa la información?
- ¿cómo se consulta o persiste?
- ¿qué plantilla muestra el resultado?
- ¿qué controles transversales aplica el proyecto?

Gracias
¿Preguntas?

